



GitLab_CheatSheet

Cheatsheet for DevOps Engineers

Repositories:

Command	Description
<code>git init</code>	Initialize a new repository
<code>git clone <repository></code>	Clone a repository to your local machine.
<code>git remote add origin <repository></code>	Add a remote repository.
<code>git push -u origin <branch></code>	Push changes to a branch.
<code>git pull</code>	Pull changes from a remote repository.
<code>git status</code>	Check the status of your local repository.
<code>git log</code>	View commit history.

Synchronizing Repositories:

Command	Description
<code>git fetch <remote></code>	Fetch changes from the remote but do not update tracking branches.
<code>git fetch --prune <remote></code>	Delete remote refs removed from the remote repository.
<code>git remote -v</code>	List all remote repositories
<code>git remote set-url origin <new-repository></code>	<code>git remote set-url origin <new-repository></code>

Branches:

Command	Description
<code>git branch</code>	List all branches
<code>git branch -a</code>	Show all branches (including remote).
<code>git branch <branch></code>	Create a new branch.
<code>git checkout <branch></code>	Switch to a different branch.
<code>git merge <branch></code>	Merge a branch into the current branch.
<code>git branch -d <branch></code>	Delete a branch.
<code>git branch -m <old-branch> <new-branch></code>	Rename a branch.

Commits:

Command	Description
<code>git add <file></code>	Add a file to the staging area.
<code>git commit -m "<message>"</code>	Commit changes with a message.
<code>git commit --amend</code>	Amend the last commit.
<code>git reset <file></code>	Unstage a file.
<code>git reset --hard</code>	Discard all changes.
<code>git diff <file></code>	Show changes between commits.
<code>git diff --staged <file></code>	Show changes between the staging area and the repository.
<code>git blame <file></code>	Show who made changes to a file.

Tagging Commits:

Command	Description
<i>git tag</i>	<i>List all tags.</i>
<i>git tag <name> <commit sha></i>	<i>Create a tag for a specific commit.</i>
<i>git tag -a <name> <commit sha></i>	<i>Create an annotated tag.</i>
<i>git tag -d <name></i>	<i>Remove a tag from the local repository.</i>

Stashing Changes:

Command	Description
<i>git stash</i>	<i>Stash current changes.</i>
<i>git stash pop</i>	<i>Apply and clear the latest stash</i>
<i>git stash apply</i>	<i>Apply stashed changes without clearing them.</i>
<i>git stash drop</i>	<i>Delete a specific stash.</i>

Merge Requests:

Command	Description
<i>git merge --no-ff <branch></i>	<i>Merge with a new commit.</i>
<i>git cherry-pick <commit></i>	<i>Apply a specific commit to the current branch.</i>
<i>git rebase <branch></i>	<i>Rebase the current branch onto another.</i>
<i>git rebase -i <commit></i>	<i>Interactive rebase.</i>

Issues:

Command	Description
<i>git commit -m "Fixes #<issue-number>"</i>	<i>Link a commit to an issue.</i>
<i>git commit -m "Closes #<issue-number>"</i>	<i>Close an issue with a commit.</i>
<i>git commit -m "Refs #<issue-number>"</i>	<i>Reference an issue with a commit.</i>
<i>git issue list</i>	<i>List all issues</i>
<i>git issue show <issue-number></i>	<i>Show details of an issue</i>
<i>git issue create</i>	<i>Create a new issue</i>
<i>git issue edit <issue-number></i>	<i>Edit an issue</i>
<i>git issue close <issue-number></i>	<i>Close an issue</i>

Wiki Management:

Command	Description
<i>git wiki list</i>	<i>List all wiki pages.</i>
<i>git wiki create <page></i>	<i>Create a new wiki page.</i>
<i>git wiki edit <page></i>	<i>Edit a wiki page.</i>
<i>git wiki delete <page></i>	<i>Delete a wiki page.</i>
<i>git wiki show <page></i>	<i>Show the contents of a wiki page</i>

CI/CD:

Command	Description
<code>.gitlab-ci.yml</code>	Configuration file for CI/CD.
<code>gitlab-runner register</code>	Register a runner.
<code>gitlab-runner run</code>	Run a runner.
<code>gitlab-runner exec</code>	Execute a job locally.
<code>gitlab-runner verify</code>	Verify runner configuration.
<code>gitlab-runner uninstall</code>	Uninstall a runner.
<code>gitlab-runner list</code>	List all runners

Groups:

Command	Description
<code>git group list</code>	List all groups
<code>git group show <group></code>	Show details of a group
<code>git group create <group></code>	Create a new group
<code>git group edit <group></code>	Edit a group
<code>git group delete <group></code>	Delete a group

Settings:

Command	Description
<code>git config --global user.name "<name>"</code>	Set your name
<code>git config --global user.email "<email>"</code>	Set your email
<code>git config --global core.editor "<editor>"</code>	Set your default editor
<code>git config --global color.ui true</code>	Enable colored output
<code>git config --global alias.<alias-name> "<command>"</code>	Create an alias for a command

Best Practices:

1. Use meaningful branch names (e.g., feature/login, bugfix/payment-issue).
2. Write concise and descriptive commit messages.
3. Regularly pull from the main branch to avoid merge conflicts.
4. Use '.gitignore' to manage ignored files efficiently.
5. Leverage CI/CD pipelines for automated testing and deployment.

Example: .gitlab-ci.yml File

stages:

- build
- test
- deploy

build:

stage: build

script:

- echo "Building the application..."

unit_test:

stage: test

script:

- echo "Running unit tests..."

deploy:

stage: deploy

script:

- echo "Deploying the application..."

Common Errors and Troubleshooting:

1. Authentication Error:

- Ensure SSH keys or access tokens are correctly configured.

2. Merge Conflicts:

- Use `git merge --abort` to cancel a merge.
- Resolve conflicts manually, then `git add` and `git commit`.

3. Detached HEAD:

- Use `git checkout <branch>` to return to a branch.

Security and Permissions:

- Set up personal access tokens for secure authentication.
- Use role-based access control (RBAC) for managing team permissions.